Visual representation of how each merge strategy restructures your Git history.

In these diagrams:

- **A and B** represent the base commits on the main branch.
- **X and Y** represent the work-in-progress commits on your feature branch.
- **M** represents a merge commit.

1. Basic Merge (No Fast-Forward)

Preserves the exact history by keeping the feature branch path separate and connecting them via a dedicated merge commit M.

text

```
   X --- Y        (Feature Branch)
  /     \
A -- B ------- M      (main Branch)
```

2. Squash Merge

Combines X and Y into a single, brand-new commit (XY). The feature branch history is dropped on main.

text

```
A -- B -- XY         (main Branch)
```

3. Rebase and Fast-Forward

Moves your feature commits X and Y so they sit directly on top of the latest commit B. They are rewritten as X' and Y' in a perfectly straight line.

text

```
A -- B -- X' -- Y'     (main Branch)
```

4. Semi-Linear Merge

First, the feature branch is rebased on top of B (becoming X' and Y'), and then it is merged with a non-fast-forward merge commit M. This keeps the history linear but clearly shows where the feature container starts and ends.

text

```
      X' --- Y'    (Rebased Feature)
     /      \
```

A ------ B -------- M (main Branch)

When a merge conflict occurs, it means both the main branch (at commit B) and your feature branch (at commit X or Y) modified the exact same line of code.

The biggest difference between these strategies is **when** and **how many times** you have to resolve that conflict.

1. Basic Merge & Squash Merge (The One-Time Fix)

Both of these strategies handle conflicts the exact same way: **You resolve all conflicts exactly once at the very end.**

- **The Process:** Git compares the final state of your feature branch (Y) with the final state of main (B). It highlights the overlapping lines, you fix them, and you commit the result.
- **Visualizing the Conflict Resolution:**

text

Basic Merge:

```
   X --- Y
  /     \ <-- You resolve conflicts right here, creating M
A -- B ------- M
```

Squash Merge:

A -- B -- XY <-- You resolve conflicts right here, creating XY

2. Rebase & Semi-Linear Merge (The Commit-by-Commit Fix)

Because both of these strategies use git rebase under the hood, **you resolve conflicts iteratively, commit by commit.** Git replays your feature branch history onto main one step at a time.

- **The Process:**
    1. Git tries to move commit X onto B. If X conflicts with B, **Git pauses.**
    2. You fix the conflict for X and type git rebase --continue. This creates X'.
    3. Git then tries to move commit Y onto X'. If Y also conflicts, **Git pauses again.**
    4. You fix the conflict for Y and type git rebase --continue. This creates Y'.
- **Visualizing the Conflict Resolution:**

text

Step 1: Git pauses at X. You fix conflicts here to create X'.

```
    [X] <-- Conflicting? Fix it.
   /
```

A ------ B

Step 2: Git moves to Y. You fix conflicts here to create Y'.

[Y] <-- Conflicting? Fix it again.

/

A ------ B -- X'

Final Result (Rebase & FF):

A ------ B -- X' -- Y'

Final Result (Semi-Linear):

X' -- Y'

/ \ <-- Standard merge happens automatically now (no conflicts left!)

A ------ B ------- M

Summary of the Trade-Offs

Strategy	**Conflict Frequency**	**Risk / Complexity**
Basic / Squash	**Once.** You only look at the final code differences.	**Low.** Simple to reason about, but can result in giant, messy conflict blocks if the branches diverged heavily.
Rebase / Semi-Linear	**Potentially Multiple Times.** You may have to fix conflicts for *every single commit* in your feature branch.	**Higher.** If you have 10 WIP commits that touch the same file, you might have to resolve the same conflict 10 times in a row unless you squash before rebasing.